CSA0823-Python Programming(Slot C)

Name: D.S.THARUNIYA PRIYA

Reg.No: 192511317

Assigned Date: 01-10-2026

Submitted Date: 02-10-2026

Python Data Type, Variable, Function Program Questions

1. Swap without a third variable

Two variables a = 10 and b = 25 are given. Swap their values without using a temporary variable. Then explain why your method works.

Logic

- Get two numbers a and b from the user.
- Swap the values using a, b = b, a.
- If a = 10 and b = 25, then a = 25 and b = 10.

Sample Input

Enter a: 10

Enter b: 25

Sample Output

Before swapping:

a = 10

b = 25

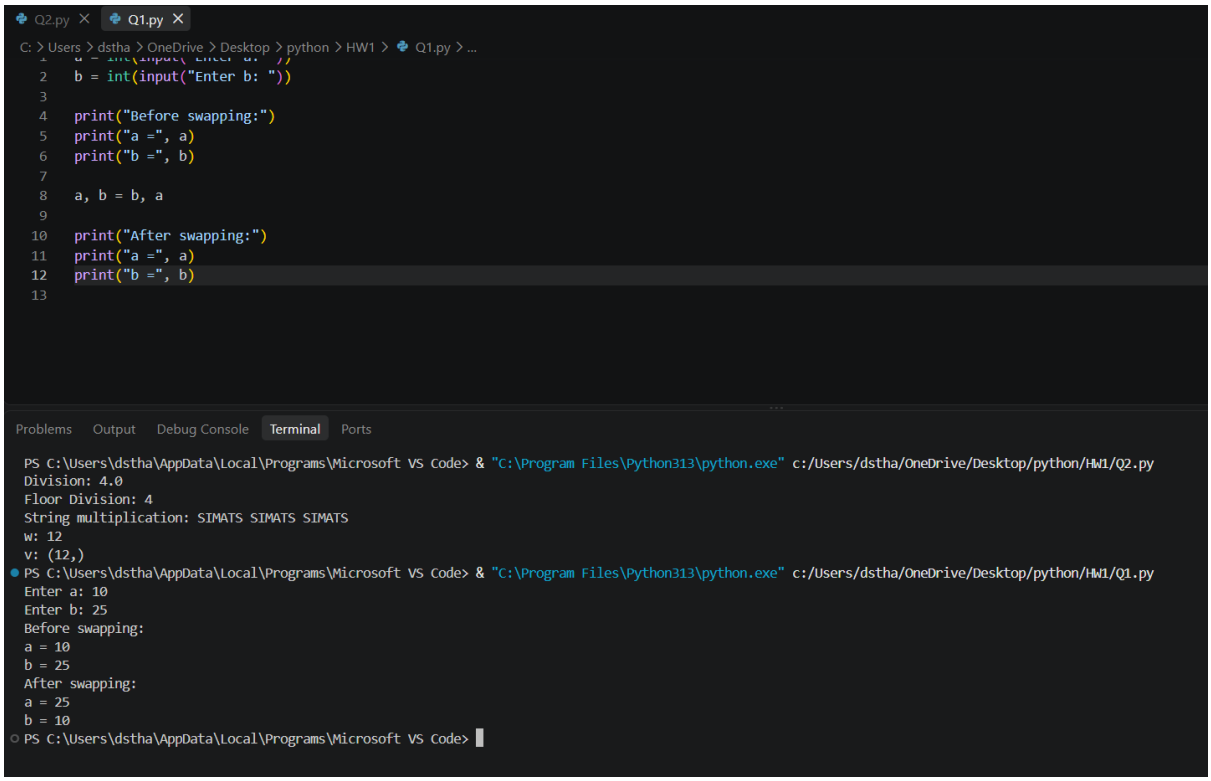
After swapping:

a = 25

b = 10

Result

The values of two variables were successfully swapped without using a third variable.



2. Predict the type
Without running the code, write the data type of each result, then verify:

python

```
x = 5 / 2

y = 5 // 2

z = "5" * 3

w = (5)

v = (5,)
```

Which ones surprised you, and why?

Logic

- Get a number from the user and use it in the given operations.
- / gives the division result as a float.
- // gives the floor division result as an int.
- Multiplying a string repeats it.
- (5) is an int, while (5,) is a tuple.
- Example: 5 / 2 → 2.5 → float.

Sample Input

Enter a number: 5

Enter a divisor: 2

Enter a string: 5

Enter repetition count: 3

Sample Output

Division: 2.5

Type: float

Floor Division: 2

Type: int

String multiplication: 555

Type: str

w = 5

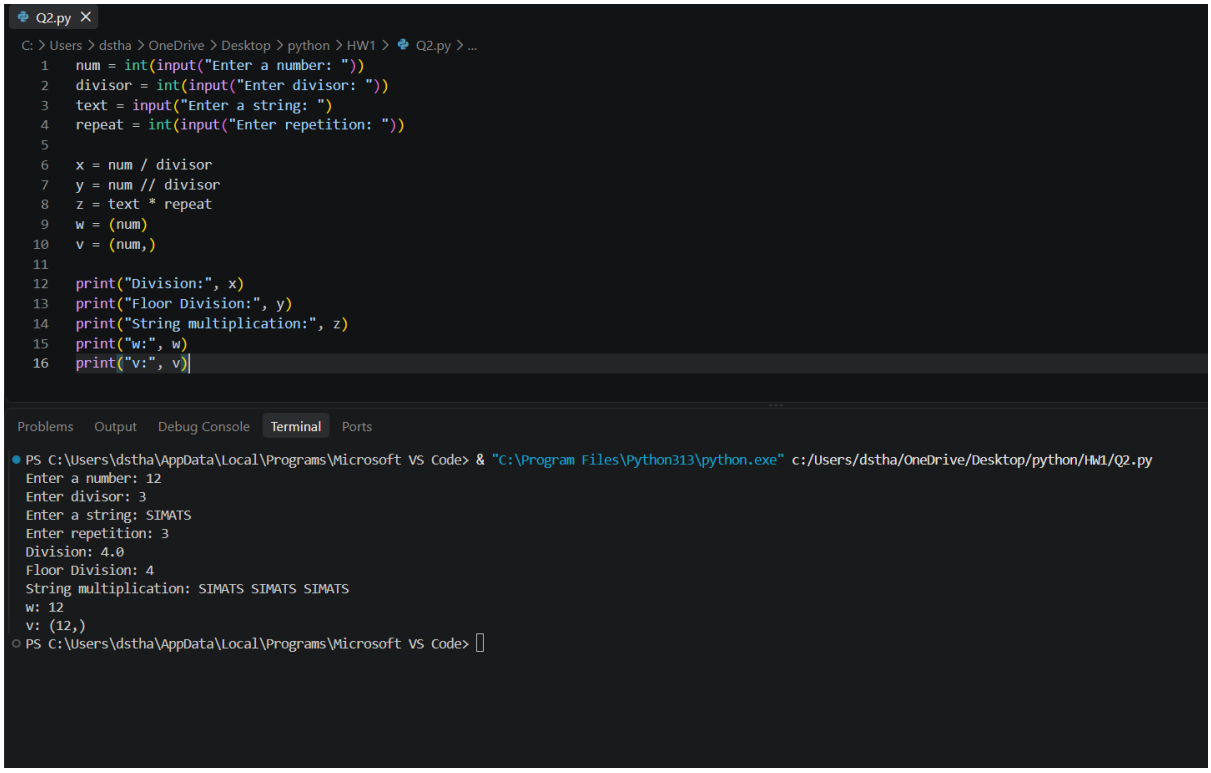
Type: int

v = (5,)

Type: tuple

Result

The values and their corresponding data types were successfully displayed.



3. The mutable trap

python

a = [1, 2, 3]

b = a

b.append(4)

print(a)

Predict the output. Then change the code so that a stays unchanged. Explain the difference between assignment and copying.

Logic

- Create a list a = [1, 2, 3].
- Write b = a.
- Both a and b refer to the same list.
- b.append(4) adds 4 to that shared list.
- Therefore a also changes.
- Output → [1, 2, 3, 4].
- To keep a unchanged, create a copy using b = a.copy().
- Now a and b are separate lists.
- b.append(4) changes only b.

Sample Input

a = [1, 2, 3]

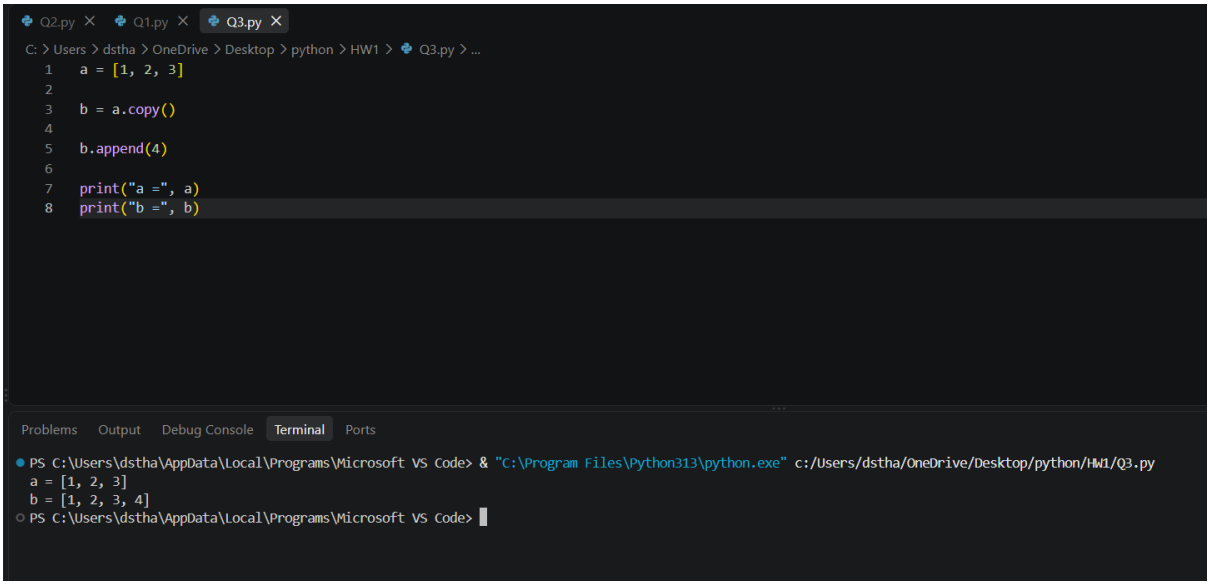
Sample Output

Original list a: [1, 2, 3]

New list b: [1, 2, 3, 4]

Result

A copy of the list was created, so modifying b did not change a.


Q2.py
Q1.py
Q3.py
C: > Users > dstha > OneDrive > Desktop > python > HW1 > Q3.py > ...
1 a = [1, 2, 3]
2
3 b = a.copy()
4
5 b.append(4)
6
7 print("a =", a)
8 print("b =", b)
Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python313\python.exe" c:/Users/dstha/OneDrive/Desktop/python/HW1/Q3.py
a = [1, 2, 3]
b = [1, 2, 3, 4]
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code>

4. Temperature converter
Write a function convert(temp, unit) that converts Celsius to Fahrenheit if unit is "C", and Fahrenheit to Celsius if unit is "F". What should it do if the unit is anything else? Decide and justify.

Logic

- Define a function convert(temp, unit) with temperature and unit as parameters.
- If the unit is "C", convert Celsius to Fahrenheit.
- Formula: F = (C * 9 / 5) + 32.
- If the unit is "F", convert Fahrenheit to Celsius.
- Formula: C = (F - 32) * 5 / 9.
- If the user enters any other unit, display "Invalid unit".
- The if, elif, and else statements are used to select the correct conversion.

Sample Input

Enter temperature: 25

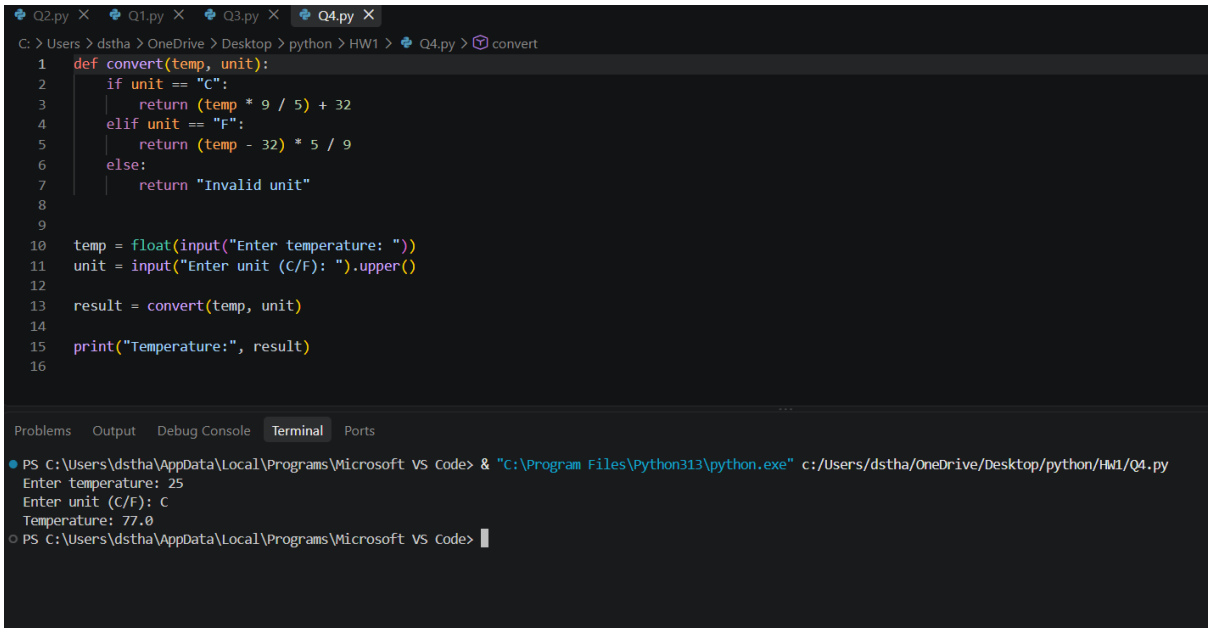
Enter unit (C/F): C

Sample Output

Temperature: 77.0 F

Result

The temperature was successfully converted according to the selected unit, and invalid units were handled without performing a conversion.


Q2.py
Q1.py
Q3.py
Q4.py
C: > Users > dstha > OneDrive > Desktop > python > HW1 > Q4.py > convert
def convert(temp, unit):
if unit == "C":
return (temp * 9 / 5) + 32
elif unit == "F":
return (temp - 32) * 5 / 9
else:
return "Invalid unit"
temp = float(input("Enter temperature: "))
unit = input("Enter unit (C/F): ").upper()
result = convert(temp, unit)
print("Temperature:", result)
Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python313\python.exe" c:/Users/dstha/OneDrive/Desktop/python/HW1/Q4.py
Enter temperature: 25
Enter unit (C/F): C
Temperature: 77.0
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code>

5. String detective

Take any sentence typed by the user. Write a program (using variables and string methods) that prints the number of words, the longest word, and the sentence in reverse word order.

Logic

- Take a sentence from the user and store it in sentence.
- Use split() to divide the sentence into individual words.
- len(words) gives the number of words.
- Use max(words, key=len) to find the word having the greatest length.
- Use words[::-1] to reverse the order of the words.
- Use " ".join() to combine the reversed words into a sentence.

Sample Input

Enter a sentence: Python programming is very interesting

Sample Output

Number of words: 5

Longest word: programming

Reverse word order: interesting very is programming Python

Result

The program successfully counted the words, identified the longest word, and displayed the sentence in reverse word order.

Q2.py Q1.py Q3.py Q4.py Q5.py

C: > Users > dstha > OneDrive > Desktop > python > HW1 > Q5.py > sentence

```python
sentence = input("Enter a sentence: ")

words = sentence.split()

word_count = len(words)
longest_word = max(words, key=len)
reverse_words = words[::-1]

print("Number of words:", word_count)
print("Longest word:", longest_word)
print("Reverse word order:", " ".join(reverse_words))

```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python313\python.exe" c:/Users/dstha/OneDrive/Desktop/python/HW1/Q5.py
Enter a sentence: PYHTON IS A INTERESTING LANGUAGE
Number of words: 5
Longest word: INTERESTING
Reverse word order: LANGUAGE INTERESTING A IS PYHTON
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code>
```

6. Fix the scope bug

python

```
count = 0

def increase():
    count = count + 1

increase()

print(count)
```

This code gives an error. Find out why, then fix it in **two different ways**. Which way is better practice?

Logic

- count = 0 is created outside the function, so it is a global variable.
- Inside increase(), Python assumes count is a local variable when we try to assign to it.
- Therefore count = count + 1 causes an error because Python tries to use the local count before giving it a value.
- **Method 1:** Use the global keyword to tell Python that count refers to the global variable.
- **Method 2:** Avoid changing a global variable and instead pass the value to the function and return the updated value.
- The second method is generally better practice because it avoids unnecessary global variables and makes the function easier to understand and reuse.

Sample Input

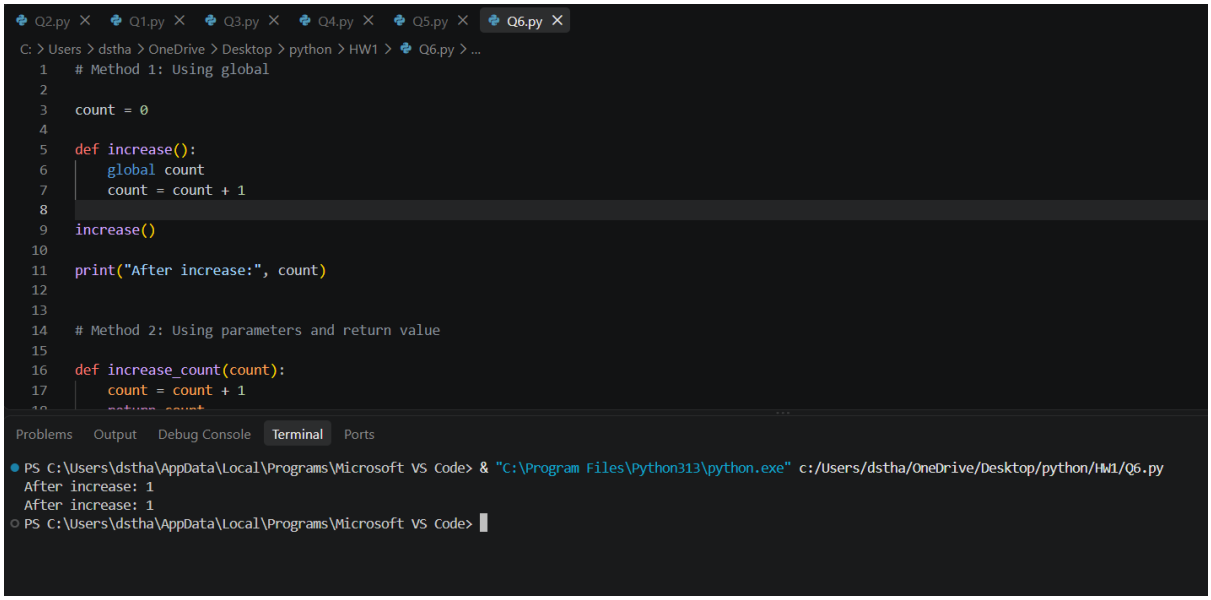
Initial count = 0

Sample Output

After increase: 1

Result

The scope error was successfully fixed using two methods. Passing and returning the value is better practice than depending on a global variable.


Q2.py
Q1.py
Q3.py
Q4.py
Q5.py
Q6.py
C: > Users > dstha > OneDrive > Desktop > python > HW1 > Q6.py > ...
Method 1: Using global
count = 0
def increase():
global count
count = count + 1
increase()
print("After increase:", count)
Method 2: Using parameters and return value
def increase_count(count):
count = count + 1
Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python313\python.exe" c:/Users/dstha/OneDrive/Desktop/python/HW1/Q6.py
After increase: 1
After increase: 1
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code>

7. Function with a default surprise
Write a function greet(name, greeting="Hello") and call it in four different ways (positional, keyword, using the default, and swapping argument order with keywords). Then write one call that would cause an error and explain why.

Logic

- Define greet(name, greeting="Hello").
- name is a required parameter, while greeting has the default value "Hello".
- Positional arguments are matched according to their order.
- Keyword arguments are matched using parameter names.
- If greeting is not provided, "Hello" is automatically used.
- Arguments can be given in a different order when keyword arguments are used.
- A call such as greet() causes an error because the required name argument is missing.

Sample Input

No user input required.

Sample Output

Hello Priya

Good Morning Priya

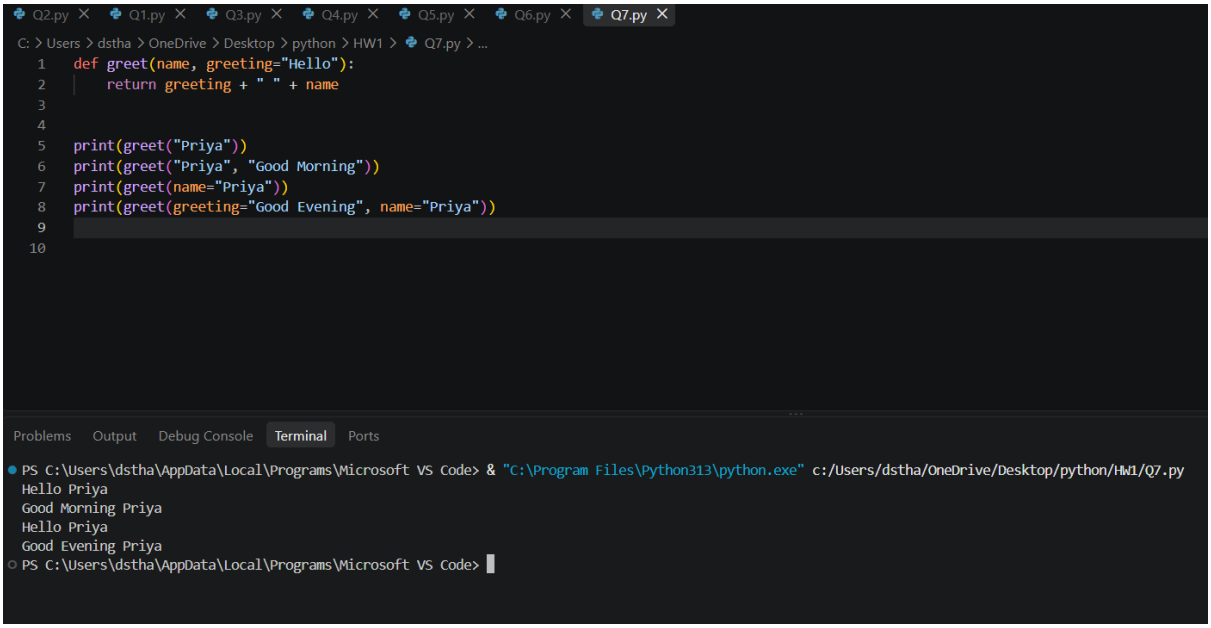
Welcome Priya

Good Evening Priya

Error: name is required

Result

The function was successfully called using positional arguments, keyword arguments, the default argument, and swapped keyword argument order. Calling greet() without name would cause a TypeError.


Q2.py
Q1.py
Q3.py
Q4.py
Q5.py
Q6.py
Q7.py
C: > Users > dstha > OneDrive > Desktop > python > HW1 > Q7.py > ...
1 def greet(name, greeting="Hello"):
2 return greeting + " " + name
3
4
5 print(greet("Priya"))
6 print(greet("Priya", "Good Morning"))
7 print(greet(name="Priya"))
8 print(greet(greeting="Good Evening", name="Priya"))
9
10
Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python313\python.exe" c:/Users/dstha/OneDrive/Desktop/python/HW1/Q7.py
Hello Priya
Good Morning Priya
Hello Priya
Good Evening Priya
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code>

8. Data type chooser
You must store each of the following in a Python program. Choose the best data type (int, float, str, list, tuple, set, dict, bool) and justify your choice:

- a student's roll number
- marks in 5 subjects
- the days of the week
- unique email IDs of club members
- a student's name along with their age and class
- whether a library book is issued

Logic

- Get the roll number from the user and store it as an int.
- Get 5 marks and store them in a list.
- Store the days of the week in a tuple.
- Get email IDs and store them in a set.
- Get the student's details and store them in a dict.
- Store whether the book is issued as True or False.

Sample Input

Enter roll number: 25

Enter 5 marks: 85 90 78 88 92

Enter email IDs: a@gmail.com b@gmail.com

Enter name: Priya

Enter age: 20

Enter class: CSE

Is book issued? True

Sample Output

Roll number: 25

Marks: [85, 90, 78, 88, 92]

Days: ('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday')

Email IDs: {'a@gmail.com', 'b@gmail.com'}

Student: {'name': 'Priya', 'age': 20, 'class': 'CSE'}

Book issued: True

Result

The required information was successfully stored using int, list, tuple, set, dict, and str data types.

Q2.py Q1.py Q3.py Q4.py Q5.py Q6.py Q7.py Q8.py

C: > Users > dstha > OneDrive > Desktop > python > HW1 > Q8.py > ...

```python
roll_number = int(input("Enter roll number: "))

marks = list(map(int, input("Enter 5 marks: ").split()))

email_ids = set(input("Enter email IDs: ").split())

name = input("Enter name: ")
age = int(input("Enter age: "))
student_class = input("Enter class: ")

student = {
    "name": name,
    "age": age,
    "class": student_class
}

book_issued = input("Is book issued? ")
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python313\python.exe" c:/Users/dstha/OneDrive/Desktop/python/HW1/Q8.py
Enter roll number: 317
Enter 5 marks: 95 98 96 98 97
Enter email IDs: 317@simats.com
Enter name: Tharuniya
Enter age: 18
Enter class: cse
Is book issued? yes
Roll number: 317
Marks: [95, 98, 96, 98, 97]
Days: ('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday')
Email IDs: {'317@simats.com'}
Student: {'name': 'Tharuniya', 'age': 18, 'class': 'cse'}
Book issued: yes
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code>
```

9. Return multiple values

Write a function analyze(numbers) that takes a list of numbers and returns the minimum, maximum, and average together. Show two different ways to receive and print the returned values. What data type does the function actually return?

Logic

- Get a list of numbers from the user.
- Find the minimum using min(numbers).
- Find the maximum using max(numbers).
- Find the average using sum(numbers) / len(numbers).
- Return all three values from the function.
- Receive the returned values separately and also as one variable.
- The returned values are stored together as a tuple.

Sample Input

Enter numbers: 10 20 30 40 50

Sample Output

Minimum: 10

Maximum: 50

Average: 30.0

Returned values: (10, 50, 30.0)

Return type: tuple

Result

The minimum, maximum, and average were successfully returned and displayed in two different ways.

Q2.py Q1.py Q3.py Q4.py Q5.py Q6.py Q7.py Q8.py q9.py

C: > Users > dstha > OneDrive > Desktop > python > HW1 > q9.py > ...

```python
def analyze(numbers):
    minimum = min(numbers)
    maximum = max(numbers)
    average = sum(numbers) / len(numbers)

    return minimum, maximum, average


numbers = list(map(int, input("Enter numbers: ").split()))

minimum, maximum, average = analyze(numbers)

print("Minimum:", minimum)
print("Maximum:", maximum)
print("Average:", average)

result = analyze(numbers)
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python313\python.exe" c:/Users/dstha/OneDrive/Desktop/python/HW1/q9.py
Enter numbers: 23 45 77
Minimum: 23
Maximum: 77
Average: 48.333333333333336
Returned values: (23, 77, 48.333333333333336)
Return type: tuple
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code>
```

10. Build a mini-calculator with validation
Write separate functions for add, subtract, multiply, and divide. Ask the user for two numbers and an operation, and call the right function. Make sure the program doesn't crash on division by zero or when the user types letters instead of numbers. Hint: think about type conversion and where it can fail.

Logic

- Get two numbers and the operation from the user.
- Use separate functions for addition, subtraction, multiplication, and division.
- Check the operation using if and elif.
- If the second number is 0 during division, display an error message.
- Use try-except to handle letters entered instead of numbers.

Sample Input

Enter first number: 20

Enter second number: 5

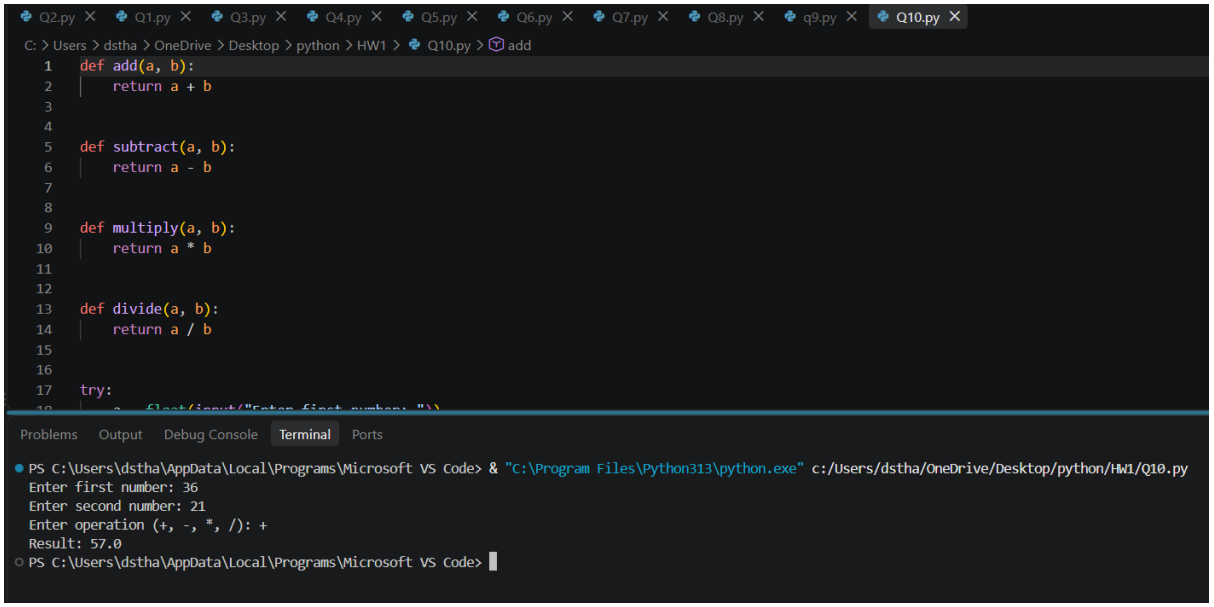
Enter operation (+, -, *, /): *

Sample Output

Result: 100.0

Result

The calculator successfully performs the selected operation and handles invalid input and division by zero.


Q2.py Q1.py Q3.py Q4.py Q5.py Q6.py Q7.py Q8.py q9.py Q10.py
C: > Users > dstha > OneDrive > Desktop > python > HW1 > Q10.py > add
1 def add(a, b):
2 return a + b
5 def subtract(a, b):
6 return a - b
9 def multiply(a, b):
10 return a * b
13 def divide(a, b):
14 return a / b
17 try:
Problems Output Debug Console Terminal Ports
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python313\python.exe" c:/Users/dstha/OneDrive/Desktop/python/HW1/Q10.py
Enter first number: 36
Enter second number: 21
Enter operation (+, -, *, /): +
Result: 57.0
PS C:\Users\dstha\AppData\Local\Programs\Microsoft VS Code>